QUESTION 1

Assumptions:

1. **Building**: Every building has a unique identifier (Building_ID) and an address.
2. **Apartment**: Apartments have unique numbers within a building, and are defined by their size (e.g., 1BR, 2BR).
3. **Tenant**: Tenants lease apartments and may rent parking spaces. Tenant data includes name and phone number.
4. **Lease**: A lease is tied to one tenant and one apartment and records the lease date and the period of the lease.
5. **Manager**: Each building has one manager, identified by name, phone number, and salary.
6. **Parking**: Buildings have parking spaces that tenants may optionally rent.

**E-R Diagram Design**

****Entities**:**

1. Building (Building_ID` (Primary Key), `Address`)

2. Apartment (Apt_ID` (Primary Key), `Apt_Number`, `Size` (e.g., 1BR, 2BR), `Building_ID` (Foreign Key))

3. Tenant (Tenant_ID` (Primary Key), `Name`, `Phone`)

4. Lease (Lease_ID` (Primary Key), `Lease_Date`, `Period`, `Apt_ID` (Foreign Key), `Tenant_ID` (Foreign Key))

5. Manager (Manager_ID` (Primary Key), `Name`, `Phone`, `Salary`, `Building_ID` (Foreign Key))

6. Parking Space (Parking_ID` (Primary Key), `Space_Number`, `Building_ID` (Foreign Key))

**Relationships:**

1. **Manages**: A building is managed by one manager. A manager may manage multiple buildings.
2. **Contains**: A building contains one or more apartments.
3. **Leases**: A tenant leases an apartment, and each lease has a date and a period.
4. **Rents Parking**: A tenant can rent parking spaces from the building.

QUESTION 2

**SQL Commands to Create Tables:**

1. `Building` Table

```
CREATE TABLE Building (
   Building_ID INT PRIMARY KEY, Address VARCHAR(255) NOT NULL
);
```

2. `Apartment` Table

```
CREATE TABLE Apartment (
   Apt_ID INT PRIMARY KEY,  Apt_Number VARCHAR(10) NOT NULL, Size VARCHAR(10) NOT NULL, Building_ID INT, FOREIGN KEY (Building_ID) REFERENCES Building(Building_ID)
);
```

3. `Tenant` Table

```
CREATE TABLE Tenant (
   Tenant_ID INT PRIMARY KEY, Name VARCHAR(100) NOT NULL,  Phone VARCHAR(15) NOT NULL
);
```

4. `Lease` Table

```
CREATE TABLE Lease (
   Lease_ID INT PRIMARY KEY, Lease_Date DATE NOT NULL, Period INT NOT NULL, Tenant_ID INT, Apt_ID INT, FOREIGN KEY (Tenant_ID) REFERENCES Tenant(Tenant_ID), FOREIGN KEY (Apt_ID) REFERENCES Apartment(Apt_ID)
);
```

5. `Parking_Space` Table

```
CREATE TABLE Parking_Space (
   Parking_ID INT PRIMARY KEY, Space_Number VARCHAR(10) NOT NULL,  Building_ID INT, FOREIGN KEY (Building_ID) REFERENCES Building(Building_ID)
);
```

6. `Rents_Parking_Space` Table

```
CREATE TABLE Rents_Parking_Space (
  Tenant_ID INT, Parking_ID INT, PRIMARY KEY (Tenant_ID, Parking_ID), FOREIGN KEY (Tenant_ID) REFERENCES Tenant(Tenant_ID),  FOREIGN KEY (Parking_ID) REFERENCES Parking_Space(Parking_ID)
);
```

7. `Manager` Table

```
CREATE TABLE Manager (
  Manager_ID INT PRIMARY KEY, Name VARCHAR(100) NOT NULL, Phone VARCHAR(15) NOT NULL,  Salary DECIMAL(10, 2) NOT NULL
);
```

8. `Manages_Building` Table

```
CREATE TABLE Manages_Building (
  Manager_ID INT, Building_ID INT, PRIMARY KEY (Manager_ID, Building_ID), FOREIGN KEY (Manager_ID) REFERENCES Manager(Manager_ID),  FOREIGN KEY (Building_ID) REFERENCES Building(Building_ID)
);
```

**Constraints That Could Not Be Captured:**

1. Uniqueness of Apartment Numbers within a Building:
  - In a building, the combination of `Apt_Number` and `Building_ID` must be unique. This can't be captured by the primary key alone. It can be handled by adding a unique constraint:  ALTER TABLE Apartment ADD CONSTRAINT UNIQUE (Apt_Number, Building_ID);

2. One Manager per Building:
  - The model allows multiple managers for a single building (due to the `Manages_Building` table). If each building should have only one manager, a unique constraint should be enforced:  ALTER TABLE Manages_Building ADD CONSTRAINT UNIQUE (Building_ID);

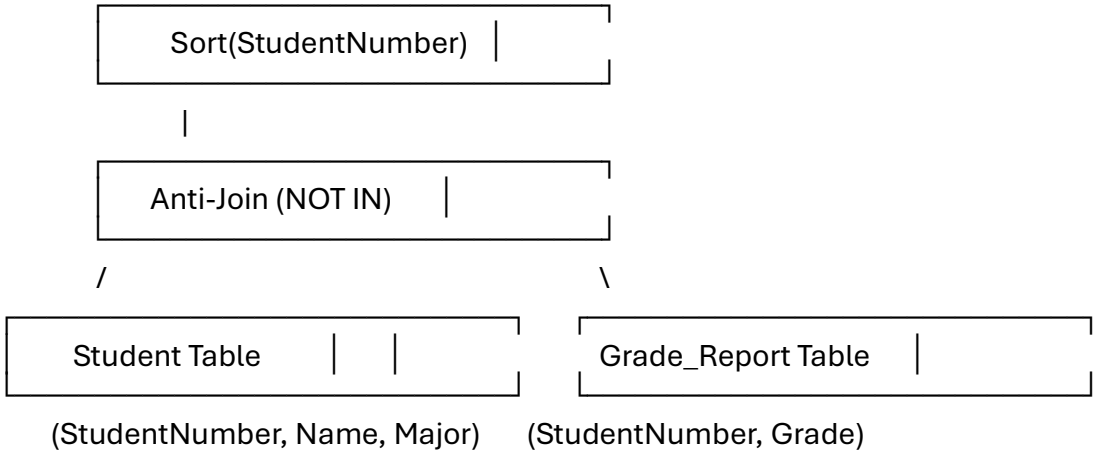
3. Lease Period Validation:
  - The duration of the lease period is stored as an integer, but no specific constraint checks the length of the lease period, such as minimum or maximum lease terms. This could be enforced via application logic or a check constraint:  ALTER TABLE Lease ADD CONSTRAINT CHECK (Period > 0);

QUESTION 3

**(a) SQL Query:**

```
SELECT DISTINCT S.StudentNumber, S.Name, S.Major
FROM Student S
WHERE S.StudentNumber NOT IN (
   SELECT GR.StudentNumber
   FROM Grade_Report GR
   WHERE GR.Grade IN ('D', 'F')
)
ORDER BY S.StudentNumber ASC;
```

**(b) Query Tree for the SQL Query:**




(c) Join Algorithm: Nested-Loops vs. Sort-Merge Join

Join Type: Anti-join (NOT IN)

Nested-Loops Join: In a nested-loop join, for each row in the outer table (in this case, `Student`), the system checks each row in the inner table (in this case, the result of the subquery on `Grade_Report`).
**Advantages**: Simple to implement and works well with smaller datasets, or if one of the tables is small and indexed.
**Disadvantages**: Can be slow with large datasets since it requires multiple scans over the inner table for every row in the outer table.

Sort-Merge Join: In a sort-merge join, both tables are sorted based on the join attribute (`StudentNumber`), and then the sorted results are merged in a single pass.
Advantages: Performs better than nested-loop when both tables are already sorted or can be sorted easily. If both tables are large, this can be more efficient than nested-loops.
-Disadvantages: Requires both tables to be sorted first, which can add overhead if they are not already sorted.

Since we are dealing with a "NOT IN" scenario (anti-join), the sort-merge join may be preferable **if** the `Grade_Report` table is large and requires significant processing. If both tables can be sorted efficiently, a sort-merge join can reduce the number of comparisons needed.
On the other hand, if the `Grade_Report` is relatively small or already indexed, a nested-loops join might be more efficient as there would be fewer comparisons needed.

(d) Index to Improve Performance:

If I were to create one index to improve the performance of the join, I would create an index on the `StudentNumber` column of the `Grade_Report` table:

```sql
CREATE INDEX idx_grade_report_student_number ON Grade_Report(StudentNumber);
```
**Reasoning**:
- The query checks for students in the `Grade_Report` table with grades of 'D' or 'F'. This index would speed up the filtering of students who have received these grades.
- By indexing `StudentNumber` in `Grade_Report`, we make the anti-join process faster because the subquery (which checks for students with D or F grades) will be executed more efficiently.
- The index will reduce the time spent scanning the `Grade_Report` table when performing the "NOT IN" operation in the main query.

(e) Advantages and Disadvantages of an Index on `StudentNumber`:

Advantages:
1. **Faster Query Execution**:
   - Indexing the `StudentNumber` on the `Student` table or the `Grade_Report` table would make retrievals based on `StudentNumber` faster, improving the query's overall performance.
Disadvantages:
Increased Storage:
   - Creating an index on `StudentNumber` will consume additional disk space, as the database needs to store the index alongside the actual data.

QUESTION 4

**(a)**

```
SELECT c.name, c.city, c.country, c.rating
FROM customer c
WHERE (
   SELECT COUNT(*)
   FROM "order" o
   WHERE o.customer_id = c.id
) > (
   SELECT AVG(order_count)
   FROM (
      SELECT COUNT(*) AS order_count
      FROM "order"
      GROUP BY customer_id
   ) AS avg_orders
);
```

**(b)**

```
SELECT d.name
FROM dept d
WHERE EXISTS (
   SELECT 1
   FROM sales_rep s
   WHERE s.dept_id = d.id
);
```

**(c)**

```
SELECT s.first_name, s.last_name
FROM sales_rep s
WHERE NOT EXISTS (
   SELECT 1
   FROM customer c
   WHERE c.sales_rep_id = s.id
);
```

**(d)**

```
SELECT DISTINCT c.country
FROM customer c
WHERE c.country NOT IN (
   SELECT DISTINCT c2.country
   FROM customer c2
   JOIN sales_rep s ON c2.sales_rep_id = s.id
);
```

QUESTION 5

**(a)**
A line with no arrow** in an ER model represents a relationship between entities. It indicates a many-to-many** relationship, meaning that multiple instances of one entity can be associated with multiple instances of the other entity.

**(b)**
A **weak entity** is an entity that cannot be uniquely identified by its attributes alone. It depends on a "strong" or "owner" entity for identification.

A **partial key** is an attribute or set of attributes of a weak entity that can uniquely identify it, but only when combined with the primary key of its related strong entity.

**(c)**
A **primary key constraint** ensures that a column or set of columns uniquely identifies each row in a table. The values must be unique and non-null.

A **foreign key constraint** ensures that the value in one table corresponds to a value in the primary key of another table, enforcing referential integrity between the two tables.

**(d)**
In "**cascading delete**," deletion of a referenced tuple causes deletion of all referencing tuples. Why does it not happen the other way around?
In a **cascading delete**, deleting a referenced tuple (parent) causes all referencing tuples (children) to be deleted to maintain referential integrity. It doesn't happen the other way around because a referencing tuple (child) depends on the existence of the referenced tuple (parent). Deleting the child does not affect the existence of the parent.

(e)
**Relational calculus (declarative):** You specify what you want from the database without stating how to compute it. The focus is on the result.
**Relational algebra (procedural)**: You specify how to obtain the result step by step, defining the operations needed to retrieve the desired data.

(f)
A view is a virtual table based on the result of a query. It doesn't store data itself but provides a way to look at data from one or more tables.
Views support logical data independence** by allowing users to interact with the data without being aware of changes to the underlying table structures. Views can present a consistent interface even if the schema changes.

QUESTION 6

**(a) Path Taken When Searching for Key 105:**

1. Start at Root Node (B1: 25, 144):
   - Compare 105 with 25 and 144.
   - Since ( 25 < 105 < 144 ), traverse to Child Node B3.

2. Move to Child Node B3 (64, 100):
   - Compare 105 with 64 and 100.
   - Since ( 100 < 105), we go to the next child node, which leads to the leaf nodes.

3. Reach Leaf Node B9 (100, 121):
   - Check the leaf nodes linked from B3:
   - Compare 105 with keys in B9.
   - Since ( 100 < 105 < 121 ), we find the key in this node.

**(b) Steps to Retrieve All Records in the Range 30 through 150:**

1. **Start at Root Node (B1: 25, 144):
   - Search for the key 30.
   - Since ( 25 < 30 < 144 ), traverse to Child Node B3.

2. Move to Child Node B3 (64, 100):
   - Check the leaf nodes linked to B3:
     - **Leaf Node B7**: `[25, 36, 49]`
     - **Leaf Node B8**: `[64, 81]`
     - **Leaf Node B9**: `[100, 121]`

3. Collecting Values:
   - Start from B7:
     - Collect 36, 49
   - Move to B8:
     - Collect 64, 81
   - Move to B9:
     - Collect 100, 121
   - Move to B10 (under B4):
     - Collect 144
   - Stop before reaching B11, as it contains values greater than 150.

Final Result: Keys collected in the range [30, 150]: 36, 49, 64, 81, 100, 121, 144.

**(c) Clustering Cost Impact:**
As previously discussed, a clustered B+ tree would allow for efficient sequential access to records, leading to lower I/O costs for range queries compared to an unclustered B+ tree where data may be scattered across different disk locations.

**(d) Join Selection Using Unclustered Index:**
For joining Reserves and Sailors on the `Sid` attribute, an Index Nested-Loops Join is still recommended due to the sorted nature of Sailors and the unclustered index on Sid in Reserves. This choice minimizes disk access and speeds up the join operation.